

# Tema 2.1

## 1. Clasificación de tecnologías front-end

| Categoría                                    | Elementos                                          |
|----------------------------------------------|----------------------------------------------------|
| Alternativas históricas obsoletas            | Adobe Flash (ActionScript), Java Applets, VBScript |
| Estándar nativo universal                    | JavaScript (ECMAScript)                            |
| Dialectos o superconjuntos con transpilación | TypeScript, JSX                                    |

Flash y los Applets exigían instalar un plugin y VBScript solo funcionaba en Internet Explorer; los tres fracasaron al buscar un estándar común. TypeScript y JSX no son alternativas a JavaScript: se transpilan a JavaScript antes de que el navegador los ejecute.

## 2. Justificación de arquitectura

Dos ventajas de ejecutar la lógica del cliente para reducir recursos del servidor:

1. La lógica inmediata se ejecuta en la máquina del usuario. Las validaciones de formularios, los cálculos de interfaz y las ordenaciones se resuelven en el navegador, así que el servidor gasta menos CPU y memoria.
2. Solo viajan datos, no páginas enteras. Al validar en cliente, el servidor devuelve datos puros (JSON) en lugar de documentos HTML completos, lo que reduce el ancho de banda y el trabajo de renderizado.

Concepto Zero Plugins: JavaScript lo interpretan de forma nativa todos los motores modernos (V8 en Chrome y Edge, SpiderMonkey en Firefox, JavaScriptCore en Safari) sin pedir ninguna instalación. Eso resuelve el problema histórico de los Java Applets y de Flash, que exigían plugins pesados, lentos y con agujeros de seguridad.

### 3. Matriz de selección de dialectos

| Proyecto                                             | Tecnología | Justificación                                                                                                                                                                                 |
|------------------------------------------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A. Plataforma bancaria, 30 desarrolladores           | TypeScript | Requiere validación estática de datos en tiempo de desarrollo: el tipado fuerte corta el error antes de subir a producción y el compilador comprueba parámetros y retornos entre 30 personas. |
| B. Script de 50 líneas para manipular el DOM         | Vanilla JS | No justifica una fase de compilación: se ejecuta directo en el navegador, sin dependencias ni build.                                                                                          |
| C. Interfaz con componentes reutilizables en ReactJS | JSX        | Es la sintaxis de componentes de React: mezcla HTML dentro de JavaScript y Babel la transforma en llamadas JS al compilar el árbol de componentes.                                            |

### 4. ECMAScript y transpilación

Relación ECMAScript-JavaScript: ECMAScript es la especificación formal, la norma ECMA-262 de ECMA International, que define la sintaxis, los tipos y los comportamientos; JavaScript es la implementación real de esa norma. ECMAScript es el qué y JavaScript es el cómo lo ejecuta cada navegador.

Papel del transpilador (Babel): el equipo escribe con sintaxis moderna de ES6/ES2015 (let, const, clases, funciones flecha) y Babel traduce ese código a JavaScript ES5 equivalente, compatible con navegadores antiguos. El código fuente se mantiene legible y lo que llega al usuario final es una versión que cualquier motor entiende. Es un traductor de código a código, no de código a binario.

### 5. Validación sintáctica de identificadores

| Nombre         | Válido | Motivo                                                                      |
|----------------|--------|-----------------------------------------------------------------------------|
| 1erUsuario     | No     | Empieza por dígito: un identificador solo puede comenzar por letra, \$ o _. |
| _totalFactura  | Sí     | Empieza por guion bajo, permitido.                                          |
| \$elementoDOM  | Sí     | Empieza por \$, permitido.                                                  |
| precio-final   | No     | El guion es el operador de resta: se leería como precio - final.            |
| class          | No     | Palabra reservada del lenguaje.                                             |
| montoTotal2    | Sí     | Empieza por letra; los dígitos solo están prohibidos al principio.          |
| nombre usuario | No     | No puede llevar espacios; lo correcto es nombreUsuario.                     |
| function       | No     | Palabra reservada del lenguaje.                                             |

## 6. Detección de errores por sensibilidad a mayúsculas

JavaScript distingue mayúsculas de minúsculas también en las palabras reservadas y en las funciones nativas (`alert`, `console.log`). En el código del enunciado aparecen seis palabras mal escritas, aunque el enunciado hable de cuatro:

| Erróneo     | Correcto    |
|-------------|-------------|
| Function    | function    |
| Const       | const       |
| If          | if          |
| Alert       | alert       |
| Return      | return      |
| Console.log | console.log |

```
function calcularDescuento(PrecioBase) {  
  const porcentaje = .;  
  if (PrecioBase > ) {  
    alert("Descuento aplicado");  
    return PrecioBase * porcentaje;  
  }  
  return ;  
}
```

```
console.log(calcularDescuento());
```

Se abre el modal "Descuento aplicado" y la consola imprime 22.5 (150 x 0,15).

## 7. Aplicación de convenciones profesionales

| Entidad                                    | Nombre            | Convención       |
|--------------------------------------------|-------------------|------------------|
| Variable con el nombre del cliente actual  | nombreCliente     | camelCase        |
| Constante global de reintentos de conexión | MAX_REINTENTOS    | UPPER_SNAKE_CASE |
| Clase de usuario registrado                | UsuarioRegistrado | PascalCase       |
| Referencia a un botón del DOM              | \$botonEnviar     | prefijo \$       |
| Propiedad privada del token de sesión      | _tokenSesion      | prefijo _        |

## 8. Análisis del comportamiento del ASI

```
function crearConfiguracion() {  
  return  
  { modo "oscuro", puerto };  
}  
  
const config = crearConfiguracion();  
console.log(config);
```

a) Salida real.

No se imprime nada: el script entero falla al analizarse.

Uncaught SyntaxError Unexpected token ':'

No es undefined, que es lo que ocurre con la versión de un solo atributo. Al llevar dos propiedades, el bloque que sobra tras el return no se puede leer como sentencia, el error es sintáctico y ni siquiera llega a ejecutarse el `console.log`.

b) Intervención del ASI, paso a paso.

1. El motor ve `return` seguido de un salto de línea e inserta automáticamente un punto y coma: la sentencia queda como `return;` y la función devuelve `undefined`.
2. A partir de ahí, `{ modo: "oscuro", puerto: 8080 }`; ya no es un literal de objeto, sino un bloque de código.
3. Dentro de un bloque, `modo:` se interpreta como una etiqueta y después vendría la expresión `"oscuro", puerto`. Aparece un `:` que no puede estar ahí y el analizador aborta con `SyntaxError`.
4. Como el error ocurre al analizar el fichero completo, ninguna línea llega a ejecutarse.

Con una sola propiedad, el paso 3 daría una etiqueta válida y sí se imprimiría `undefined`.

c) Código corregido.

```
function crearConfiguracion() {  
  return {  
    modo "oscuro",  
    puerto  
  };  
}  
  
const config = crearConfiguracion();  
console.log(config); // { modo: 'oscuro', puerto: 8080 }
```

La llave va en la misma línea que el `return`, o la sentencia completa en una línea. Otra opción válida es construir el objeto en una variable y devolver esa variable.

## 9. Verificación en consola y ámbito

```
let totalVentas = ;  
let TotalVentas = ;  
  
console.log(totalVentas); // 500  
console.log(TotalVentas); // 1200
```

Salida:

JavaScript es case-sensitive: `totalVentas` y `TotalVentas` son dos identificadores distintos, dos posiciones de memoria independientes. Se crean las dos variables sin que la segunda sobrescriba a la primera, y cada `console.log` lee la suya. Si estuvieran escritas igual, el motor daría error por redeclaración; al diferir en la mayúscula de la T, simplemente conviven.